

Deep Neural Networks formula cheat sheet

Here is the complete and updated Deep Neural Networks formula cheat sheet

1. Neuron & Perceptron Basics

- **Linear Model / Hypothetical Function:** $\hat{y} = f(X; W, b) = \mathbf{W}^T \mathbf{X} + b$
- **Weighted Sum (Pre-activation):** $z = \sum_{i=1}^d w_i x_i + b = \mathbf{w}^T \mathbf{x}$
- **Step Activation Function:** $\hat{y} = 1$ if $z \geq 0$ else 0
- **Sign (Signum) Activation Function:** $\text{sign}(z) = +1$ if $z > 0$, 0 if $z = 0$, -1 if $z < 0$
- **Perceptron Learning Rule:** $w_{i_new} = w_{i_old} + \eta(y - \hat{y})x_i$ (or $W_{new} = W_{old} + \eta(T - \hat{y})x$)
- **Perceptron Weight Update (Batch):** $W_{new} = W_{old} - \frac{\alpha}{2M} \sum_{i=1}^M (A_i - Y_i) \cdot X_i$
- **Perceptron Bias Update (Batch):** $B_{new} = B_{old} - \frac{\alpha}{2M} \sum_{i=1}^M (A_i - Y_i)$
- **Perceptron Decision Boundary Line:** $X_2 = -(W_0/W_1)X_1 - B/W_1$

2. Regression & Evaluation Metrics

- **Mean Squared Error (MSE) / Squared Error Loss:** $J = \frac{1}{2M} \sum (\hat{y}_i - y_i)^2$ or $\frac{1}{N} \sum (\hat{y}_i - y_i)^2$
- **Gradient of MSE:** $\nabla J = \frac{1}{N} \mathbf{X}^T (\hat{\mathbf{y}} - \mathbf{y})$
- **Root Mean Squared Error (RMSE):** $\text{RMSE} = \sqrt{MSE}$
- **Mean Absolute Error (MAE):** $J = \frac{1}{N} \sum |\hat{y}_i - y_i|$
- **R^2 Score (Coefficient of Determination):** $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$
 - **Sum of Squared Residuals (SS_{res}):** Total squared residuals from model predictions, $SS_{res} = \sum_{i=1}^N (y_i - \hat{y}_i)^2$
 - **Total Sum of Squares (SS_{tot}):** Total squared deviation from the mean, $SS_{tot} = \sum_{i=1}^N (y_i - \bar{y})^2$

3. Classification & Loss Functions

- **Binary Cross-Entropy (BCE) / Logistic Loss:**
 $J = -\frac{1}{N} \sum [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$
- **Categorical Cross-Entropy (CCE) Loss:** $J = -\frac{1}{N} \sum \sum y_k \log(\hat{y}_k)$
- **Gradient of BCE (with Sigmoid):** $\frac{\partial J}{\partial W} = (\hat{y} - y)\mathbf{x}$
- **Stochastic Gradient Descent (SGD) Update:** $w(t+1) = w(t) - \eta(\hat{y}^{(i)} - y^{(i)})x^{(i)}$
- **Mini-Batch Gradient:** $\nabla W = \frac{1}{B} \mathbf{X}_B^T (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$ (where B is batch size)

4. Activation Functions & Their Derivatives

- **Identity (Regression):** $f(z) = z$ | **Derivative:** $f'(z) = 1$
- **Sigmoid (Binary Classification):** $\sigma(z) = \frac{1}{1+e^{-z}}$
 - **Derivative:** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

- **Tanh:** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
 - **Derivative:** $\tanh'(z) = 1 - \tanh^2(z)$
- **ReLU (Hidden Layers):** $\text{ReLU}(z) = \max(0, z)$
 - **Derivative:** $\text{ReLU}'(z) = 1$ if $z > 0$ else 0
- **Leaky ReLU:** $\text{LeakyReLU}(z) = \max(\alpha z, z)$ (where $\alpha \approx 0.01$)
- **Softmax (Multi-class):** $\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$
 - **Numerical Stability Fix:** $\hat{y}_k = \frac{e^{z_k - z_{\max}}}{\sum_j e^{z_j - z_{\max}}}$

5. Deep Feed-Forward Neural Networks (DFNN) & Backpropagation

- **Layer-wise Pre-activation (Forward pass):** $Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)}$
- **Layer-wise Post-activation (Forward pass):** $A^{(l)} = \sigma^{(l)}(Z^{(l)})$
- **Total Parameter Count (including biases):** $Params = \sum (n_{l-1} \cdot n_l + n_l)$ (where n is number of units per layer)
- **Output Layer Error Signal:** $\delta^{(L)} = \frac{\partial J}{\partial Z^{(L)}} = \hat{y} - y$
- **Hidden Layer Error Signal (Chain Rule):** $\delta^{(l)} = (\delta^{(l+1)}(W^{(l+1)})^T) \odot \sigma'^{(l)}(Z^{(l)})$
- **Weight Gradient:** $\frac{\partial J}{\partial W^{(l)}} = (A^{(l-1)})^T \cdot \delta^{(l)}$
- **Bias Gradient:** $\frac{\partial J}{\partial b^{(l)}} = \text{sum}(\delta^{(l)})$
- **Weight & Bias Updates:** $W^{(l)} \leftarrow W^{(l)} - \eta \cdot \frac{\partial J}{\partial W^{(l)}}$ and $b^{(l)} \leftarrow b^{(l)} - \eta \cdot \delta^{(l)}$

6. Classification Evaluation Metrics

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$
- **Precision:** $\frac{TP}{TP+FP}$
- **Recall:** $\frac{TP}{TP+FN}$
- **F1 Score:** $\frac{2 \cdot P \cdot R}{P+R}$

7. Parameter Initialization & Architecture Design

- **He Initialization (for ReLU):** $W \sim \mathcal{N}(0, \sqrt{\frac{2}{n_{in}}})$
- **Xavier/Glorot Initialization (for Sigmoid/Tanh):** Variance $\sigma^2 = \frac{2}{n_{in} + n_{out}}$
- **Architecture Depth Heuristic:** $L \leq \log_2(\frac{5N}{3D}) + 1$ (where N =samples, D =features)
- **Parameter Constraint Heuristic:** Total $Params \leq \frac{N}{5}$
- **Possible Dropout Sub-networks:** 2^n (where n is the number of nodes subject to dropout)

8. Convolutional Neural Networks (CNN)

- **Output Spatial Shape:** $out = \lfloor \frac{N-F+2P}{S} \rfloor + 1$ (where N =Input dimension, F =Filter size, P =Padding, S =Stride)

- **Trainable Parameters in a Conv Layer:**

$(\text{Filter Height} \times \text{Filter Width} \times \text{Input Channels} + 1_{\text{bias}}) \times \text{Output Channels}$